College Platform - Low Level Design (LLD)

Database Schemas (MongoDB)

1. User Schema

```
// users collection
{
  _id: ObjectId,
  email: String, // required, unique, index
  password: String, // hashed using bcrypt
  isVerified: Boolean, // default: false
  otp: String, // for email verification
  otpExpiry: Date,
  profile: {
    firstName: String, // required
    lastName: String, // required
    rollNumber: String, // unique, required
    year: Number, // 1, 2, 3, 4
    branch: String, // CSE, ECE, IT, etc.
    college: String, // default: "Your College Name"
    phoneNumber: String,
    avatar: String, // URL to profile image
    bio: String,
    linkedinUrl: String,
    githubUrl: String,
    portfolioUrl: String,
    targetCompanies: [String], // ["Google", "Microsoft", "Amazon"]
    preferredRole: String // "SDE", "Data Scientist", "Product Manager"
  },
  gamification: {
    currentStreak: Number, // default: 0
    longestStreak: Number, // default: 0
    lastActiveDate: Date,
    totalCredits: Number, // default: 0
    level: String, // "Beginner", "Intermediate", "Advanced", "Expert"
    globalRank: Number,
    collegeRank: Number,
    badges: [{
      badgeId: ObjectId, // reference to badges collection
      awardedAt: Date,
```

```
    progress: Number // for progressive badges
  }],
  achievements: [{
    achievementId: String, // "array_master", "consistent_coder"
    title: String,
    description: String,
    unlockedAt: Date,
    creditsEarned: Number
  }]
 },
 learningStats: {
  totalProblemsAttempted: Number, // default: 0
  totalProblemsSolved: Number, // default: 0
  easyProblemsSolved: Number, // default: 0
  mediumProblemsSolved: Number, // default: 0
  hardProblemsSolved: Number, // default: 0
  averageAccuracy: Number, // percentage
  totalStudyHours: Number, // in minutes
  favoriteTopics: [String],
  weakTopics: [String],
  learningVelocity: Number, // problems per week
  lastStudySession: Date
 },
 preferences: {
  preferredLanguage: String, // "cpp", "java", "python"
  learningStyle: String, // "visual", "hands-on", "reading"
  dailyGoal: Number, // problems per day
  studyReminders: Boolean, // default: true
  emailNotifications: Boolean, // default: true
  pushNotifications: Boolean // default: true
 },
 subscription: {
  tier: String, // "Bronze", "Silver", "Gold", "Platinum"
  creditsSpent: Number, // default: 0
  mentorSessionsUsed: Number, // default: 0
  mentorSessionsAllowed: Number // based on tier
 },
 createdAt: Date, // default: Date.now
 updatedAt: Date,
 lastLogin: Date
}
```

2. Learning Track Schema

```
// learning_tracks collection
{
  _id: ObjectId,
  trackId: String, // unique identifier: "dsa_beginner", "ml_intermediate"
  title: String, // "Data Structures & Algorithms - Beginner"
  description: String,
  category: String, // "DSA", "ML", "Development", "System_Design"
  difficulty: String, // "Beginner", "Intermediate", "Advanced"
  targetYear: [Number], // [1, 2] - for which year students
  estimatedDuration: Number, // in days
  prerequisiteTracks: [String], // trackIds that should be completed first
  topics: [{
    topicId: String, // "arrays", "linked_lists", "binary_trees"
    title: String, // "Arrays and Strings"
    description: String,
    order: Number, // sequence in track
    estimatedTime: Number, // in hours
    difficulty: String,
    subtopics: [{
      subtopicId: String, // "two_pointers", "sliding_window"
      title: String,
      concepts: [String], // key concepts to learn
      problems: [ObjectId], // reference to problems collection
      resources: [{
        type: String, // "video", "article", "documentation"
        title: String,
        url: String,
        duration: Number, // in minutes for videos
        difficulty: String,
        isRecommended: Boolean
      }]
    }],
    quiz: [{
      questionId: ObjectId,
      type: String, // "mcq", "coding", "theory"
      difficulty: String
    }],
    isLocked: Boolean, // default: false
    unlockCriteria: {
      prerequisiteTopics: [String],
      minAccuracy: Number, // percentage
      minProblems: Number
    }
  }],
```

```
  totalProblems: Number,
  totalQuizzes: Number,
  enrollmentCount: Number, // default: 0
  completionRate: Number, // percentage of enrolled users who completed
  averageRating: Number,
  tags: [String], // for search and filtering
  isActive: Boolean, // default: true
  createdBy: ObjectId, // admin/instructor who created
  createdAt: Date,
  updatedAt: Date
}
```

3. User Progress Schema

```
// user_progress collection
{
  _id: ObjectId,
  userId: ObjectId, // reference to users collection
  trackId: String, // reference to learning_tracks
  enrollmentDate: Date,
  lastAccessDate: Date,
  currentTopic: String, // topicId where user is currently
  currentSubtopic: String, // subtopicId where user is currently
  overallProgress: Number, // percentage of track completed
  topicProgress: [{
    topicId: String,
    subtopicProgress: [{
      subtopicId: String,
      isCompleted: Boolean,
      completionDate: Date,
      timeSpent: Number, // in minutes
      problemsAttempted: Number,
      problemsSolved: Number,
      averageAccuracy: Number,
      lastProblemSolved: String, // problem title for home page display
      notes: String // user's personal notes
    }],
    isCompleted: Boolean,
    completionDate: Date,
    totalTimeSpent: Number,
    quizScore: Number, // out of 100
    quizAttempts: Number
  }],
  studyStreak: {
```

```
  currentStreak: Number,
  longestStreak: Number,
  lastStudyDate: Date,
  streakStartDate: Date
 },
 weeklyGoals: {
  problemsGoal: Number,
  problemsSolved: Number,
  studyHoursGoal: Number,
  studyHoursCompleted: Number,
  weekStartDate: Date
 },
 milestones: [{
  milestoneId: String, // "completed_arrays", "solved_100_problems"
  achievedAt: Date,
  creditsEarned: Number,
  badgeEarned: String
 }],
 strugglingAreas: [String], // topics where accuracy < 60%
 strongAreas: [String], // topics where accuracy > 80%
 estimatedCompletionDate: Date,
 isCompleted: Boolean,
 completionDate: Date,
 finalScore: Number, // overall track score
 certificate: {
  issued: Boolean,
  issueDate: Date,
  certificateUrl: String
 }
}
```

4. Problem Schema

```
// problems collection
{
 _id: ObjectId,
 problemId: String, // unique: "two_sum", "reverse_linked_list"
 title: String, // "Two Sum"
 description: String, // full problem statement
 difficulty: String, // "Easy", "Medium", "Hard"
 topics: [String], // ["Arrays", "Hash Table"]
 subtopics: [String], // ["Two Pointers", "Sliding Window"]
 companies: [String], // ["Google", "Microsoft", "Amazon"]
 frequency: Number, // how often asked in interviews (1-10)
```

```
likes: Number, // default: 0
dislikes: Number, // default: 0

constraints: {
  timeLimit: Number, // in seconds
  memoryLimit: Number, // in MB
  inputConstraints: [String] // ["1 <= nums.length <= 10^4"]
},

examples: [{
  input: String,
  output: String,
  explanation: String
}],

hints: [String], // progressive hints

solutions: [{
  approach: String, // "Brute Force", "Optimized", "Two Pointer"
  timeComplexity: String, // "O(n)"
  spaceComplexity: String, // "O(1)"
  description: String,
  code: {
    cpp: String,
    java: String,
    python: String,
    javascript: String
  },
  isOptimal: Boolean
}],

testCases: [{
  input: String,
  expectedOutput: String,
  isHidden: Boolean, // hidden test cases for evaluation
  explanation: String
}],

editorialContent: {
  intuition: String,
  approach: String,
  complexity: String,
  followUp: [String]
},
```

```
  relatedProblems: [ObjectId], // similar problems

  aiLearningContent: {
    conceptsRequired: [String],
    commonMistakes: [String],
    teachingPoints: [String],
    analogies: [String] // for AI tutor to explain
  },

  statistics: {
    totalAttempts: Number,
    totalSolutions: Number,
    accuracyRate: Number, // percentage
    averageTimeToSolve: Number, // in minutes
    languageDistribution: {
      cpp: Number,
      java: Number,
      python: Number,
      javascript: Number
    }
  },

  isActive: Boolean, // default: true
  isPremium: Boolean, // requires credits to unlock
  unlockCriteria: {
    prerequisiteProblems: [ObjectId],
    minUserLevel: String,
    creditsRequired: Number
  },

  createdBy: ObjectId, // admin who added
  createdAt: Date,
  updatedAt: Date,
  lastModified: Date
}
```

5. User Submission Schema

```
// user_submissions collection
{
  _id: ObjectId,
  userId: ObjectId,
  problemId: ObjectId,
```

```
  submissionId: String, // unique identifier

  code: String, // user's submitted code
  language: String, // "cpp", "java", "python", "javascript"

  status: String, // "Accepted", "Wrong Answer", "Time Limit Exceeded", "Runtime Error"

  testCasesResults: [{
    testCaseIndex: Number,
    status: String, // "Passed", "Failed", "Error"
    executionTime: Number, // in ms
    memoryUsed: Number, // in KB
    input: String,
    expectedOutput: String,
    actualOutput: String,
    errorMessage: String
  }],

  executionStats: {
    totalTime: Number, // in ms
    maxMemory: Number, // in KB
    testCasesPassed: Number,
    totalTestCases: Number
  },

  submissionTime: Date,
  timeTaken: Number, // time spent on problem in minutes

  attempts: Number, // how many times user tried this problem
  isFirstAccepted: Boolean, // first successful submission

  codeMetrics: {
    linesOfCode: Number,
    cyclomaticComplexity: Number, // code complexity score
    codeQualityScore: Number // automated score (1-10)
  },

  feedback: {
    aiGeneratedFeedback: String, // suggestions for improvement
    performanceAnalysis: String,
    alternativeApproaches: [String]
  }
}
```

6. Leaderboard Schema

```
// leaderboards collection
{
  _id: ObjectId,
  type: String, // "global", "college", "weekly", "monthly", "track_specific"
  category: String, // "overall", "dsa", "ml", "development"
  timeframe: String, // "all_time", "weekly", "monthly"

  rankings: [{
    userId: ObjectId,
    rank: Number,
    score: Number, // calculated based on problems solved, accuracy, etc.
    problemsSolved: Number,
    averageAccuracy: Number,
    streak: Number,
    level: String,

    // Additional stats for detailed leaderboard
    recentActivity: String, // "Solved Binary Tree Inorder Traversal"
    lastActiveTime: Date,
    preferredLanguage: String,
    solvingSpeed: Number, // average time per problem

    // Track-specific stats
    trackProgress: Number, // if track-specific leaderboard
    trackCompletions: Number
  }],

  lastUpdated: Date,
  isActive: Boolean,

  // Leaderboard metadata
  totalParticipants: Number,
  averageScore: Number,
  topScorers: [ObjectId], // top 10 user IDs

  // For weekly/monthly leaderboards
  periodStart: Date,
  periodEnd: Date
}
```

7. Notification Schema

```
// notifications collection
{
  _id: ObjectId,
  userId: ObjectId,
  type: String, // "contest", "session", "hackathon", "achievement", "reminder"
  category: String, // "event", "social", "learning", "system"

  title: String, // "Weekly DSA Challenge"
  message: String, // detailed notification text

  eventDetails: {
    eventType: String, // "contest", "session", "hackathon"
    eventId: ObjectId, // reference to specific event
    startDate: Date,
    endDate: Date,
    topics: [String], // ["Graphs", "DP", "Recursion"]
    difficulty: String,
    registrationRequired: Boolean,
    maxParticipants: Number,
    currentParticipants: Number,

    // For mentor sessions
    mentorName: String,
    mentorRole: String, // "SDE-2 @ Google"
    sessionType: String, // "interview_experience", "technical_guidance"
    creditsRequired: Number,

    // For hackathons
    theme: String, // "AI/ML Applications"
    teamSize: Number,
    prizes: [String],
    sponsors: [String]
  },

  actionRequired: Boolean, // needs user action (register, RSVP)
  actionUrl: String, // deep link to relevant page

  isRead: Boolean, // default: false
  isImportant: Boolean, // high priority notification

  createdAt: Date,
  scheduledFor: Date, // for scheduled notifications
  expiresAt: Date, // when notification becomes irrelevant
```

```
  metadata: {
    source: String, // "system", "admin", "mentor"
    relatedEntity: String, // "track", "problem", "event"
    relatedEntityId: ObjectId
  }
}
```

API Endpoints

1. Authentication APIs

POST /api/auth/register

```
// Payload
{
  email: "student@example.com",
  password: "securePassword123",
  firstName: "John",
  lastName: "Doe",
  rollNumber: "21CSE001",
  year: 2,
  branch: "CSE"
}

// Response
{
  success: true,
  message: "Registration successful. Please verify your email.",
  data: {
    userId: "64a7b8c9d1234567890abcde",
    email: "student@example.com",
    isVerified: false,
    otpSent: true
  }
}
```

POST /api/auth/verify-otp

```
// Payload
{
  email: "student@example.com",
  otp: "123456"
}
```

```
// Response
{
  success: true,
  message: "Email verified successfully",
  data: {
    userId: "64a7b8c9d1234567890abcde",
    token: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    user: {
      id: "64a7b8c9d1234567890abcde",
      email: "student@example.com",
      profile: {
        firstName: "John",
        lastName: "Doe",
        rollNumber: "21CSE001",
        year: 2,
        branch: "CSE"
      }
    }
  }
}
```

POST /api/auth/login

```
// Payload
{
  email: "student@example.com",
  password: "securePassword123"
}

// Response
{
  success: true,
  message: "Login successful",
  data: {
    token: "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    user: {
      id: "64a7b8c9d1234567890abcde",
      email: "student@example.com",
      profile: {
        firstName: "John",
        lastName: "Doe",
        rollNumber: "21CSE001",
        year: 2,
```

```
      branch: "CSE",
      avatar: "https://example.com/avatar.jpg"
    },
    gamification: {
      currentStreak: 7,
      totalCredits: 150,
      level: "Intermediate",
      globalRank: 1247,
      collegeRank: 12
    }
  }
 }
}
```

2. Home Page APIs

GET /api/dashboard/home

```
// Headers: Authorization: Bearer <token>
// Response
{
 success: true,
 data: {
  userProgress: {
   currentSprint: {
    trackId: "dsa_intermediate",
    trackTitle: "Arrays & Strings Mastery",
    currentTopic: "sliding_window",
    currentTopicTitle: "Sliding Window Technique",
    recentlyCompleted: "Two Sum Problem",
    nextTarget: "Hash Map Patterns",
    completionPercentage: 65,
    problemsCompleted: 13,
    totalProblems: 20,
    estimatedTimeRemaining: "3 days"
   },
   dailyStreak: 7,
   longestStreak: 15,
   totalCredits: 150,
   todayGoal: {
    problemsGoal: 3,
    problemsSolved: 1,
    studyHoursGoal: 2,
    studyHoursCompleted: 0.5
```

```
  }
},

notifications: [
  {
    id: "64a7b8c9d1234567890abcd1",
    type: "contest",
    title: "Weekly DSA Challenge",
    message: "Registration closes in 2 days",
    eventDetails: {
      startDate: "2024-08-27T10:00:00Z",
      topics: ["Graphs", "DP", "Recursion"],
      duration: 120,
      maxParticipants: 500,
      currentParticipants: 234
    },
    isImportant: true,
    actionRequired: true,
    actionUrl: "/contests/weekly-dsa-challenge",
    createdAt: "2024-08-25T08:00:00Z"
  },
  {
    id: "64a7b8c9d1234567890abcd2",
    type: "session",
    title: "Google Interview Experience",
    message: "By Rohit Sharma (SDE-2 @ Google)",
    eventDetails: {
      startDate: "2024-08-30T15:00:00Z",
      mentorName: "Rohit Sharma",
      mentorRole: "SDE-2 @ Google",
      sessionType: "interview_experience",
      creditsRequired: 25,
      maxParticipants: 50,
      currentParticipants: 12
    },
    actionRequired: true,
    actionUrl: "/sessions/google-interview-experience",
    createdAt: "2024-08-25T09:00:00Z"
  },
  {
    id: "64a7b8c9d1234567890abcd3",
    type: "hackathon",
    title: "Build for Tomorrow",
    message: "Registration starts in 1 week",
```

```
    eventDetails: {
      startDate: "2024-09-01T00:00:00Z",
      endDate: "2024-09-03T23:59:59Z",
      theme: "AI/ML Applications",
      teamSize: 4,
      registrationRequired: true
    },
    actionRequired: false,
    createdAt: "2024-08-25T10:00:00Z"
  }
],

leaderboard: {
  type: "weekly",
  userRank: 3,
  topRankers: [
    {
      userId: "64a7b8c9d1234567890abcd4",
      rank: 1,
      name: "Priya Singh",
      score: 245,
      problemsSolved: 15,
      preferredLanguage: "Java",
      avatar: "https://example.com/priya.jpg"
    },
    {
      userId: "64a7b8c9d1234567890abcd5",
      rank: 2,
      name: "Arjun Kumar",
      score: 230,
      problemsSolved: 12,
      preferredLanguage: "C++",
      avatar: "https://example.com/arjun.jpg"
    },
    {
      userId: "64a7b8c9d1234567890abcde",
      rank: 3,
      name: "You",
      score: 180,
      problemsSolved: 10,
      preferredLanguage: "Python",
      avatar: "https://example.com/your-avatar.jpg",
      isCurrentUser: true
    }
```

```
  ],
  totalParticipants: 156,
  lastUpdated: "2024-08-25T12:00:00Z"
},

quickActions: [
  {
    title: "Continue Learning",
    description: "Resume Sliding Window Technique",
    actionUrl: "/tracks/dsa_intermediate/sliding_window",
    icon: "play-circle",
    priority: 1
  },
  {
    title: "Take Mock Test",
    description: "Arrays & Strings Assessment",
    actionUrl: "/assessments/arrays-strings-mock",
    icon: "clipboard-check",
    priority: 2
  },
  {
    title: "Join Study Group",
    description: "Advanced DP Masters (4/6 members)",
    actionUrl: "/community/study-groups/advanced-dp",
    icon: "users",
    priority: 3
  },
  {
    title: "Ask Mentor",
    description: "1 session remaining this month",
    actionUrl: "/mentorship/book-session",
    icon: "message-circle",
    priority: 4
  }
],

recentActivity: [
  {
    type: "problem_solved",
    title: "Solved 'Longest Palindromic Substring'",
    timestamp: "2024-08-25T11:30:00Z",
    metadata: {
      difficulty: "Medium",
      timeTaken: 45,
```

```
        accuracy: "First Try"
      }
    },
    {
      type: "session_attended",
      title: "Attended 'System Design Basics' session",
      timestamp: "2024-08-24T16:00:00Z",
      metadata: {
        duration: 90,
        rating: 5
      }
    },
    {
      type: "helped_junior",
      title: "Helped junior with 'Binary Trees' concept",
      timestamp: "2024-08-24T14:20:00Z",
      metadata: {
        creditsEarned: 5
      }
    }
  ]
 }
}
```

GET /api/dashboard/weekly-stats

```
// Headers: Authorization: Bearer <token>
// Response
{
 success: true,
 data: {
  weekStart: "2024-08-19T00:00:00Z",
  weekEnd: "2024-08-25T23:59:59Z",
  problemsSolved: 12,
  studyHours: 18,
  conceptsLearned: ["Sliding Window", "Two Pointers", "Hash Maps"],
  accuracyTrend: [85, 78, 82, 90, 87, 92, 89], // daily accuracy
  dailyActivity: [2, 1, 3, 2, 2, 1, 1], // problems per day
  strongestTopic: "Arrays",
  weakestTopic: "Dynamic Programming",
  improvementSuggestions: [
    "Focus more on DP problems",
    "Practice contest-style problems",
    "Improve time complexity analysis"
```

```
    ]
  }
}
```

3. Learning Tracks APIs

GET /api/tracks

```
// Query Parameters: ?category=DSA&difficulty=Intermediate&year=2
// Response
{
  success: true,
  data: {
    tracks: [
      {
        trackId: "dsa_intermediate",
        title: "Data Structures & Algorithms - Intermediate",
        description: "Master intermediate DSA concepts with hands-on practice",
        category: "DSA",
        difficulty: "Intermediate",
        targetYear: [2, 3],
        estimatedDuration: 45,
        totalProblems: 120,
        totalQuizzes: 15,
        enrollmentCount: 234,
        completionRate: 78,
        averageRating: 4.6,

        userProgress: {
          isEnrolled: true,
          overallProgress: 65,
          currentTopic: "sliding_window",
          estimatedCompletionDate: "2024-09-10T00:00:00Z"
        },

        prerequisites: ["dsa_beginner"],
        tags: ["algorithms", "data-structures", "interview-prep"],

        thumbnail: "https://example.com/dsa-intermediate.jpg",
        instructorInfo: {
          name: "Dr. Sarah Johnson",
          designation: "Senior SDE @ Microsoft",
          avatar: "https://example.com/instructor.jpg"
        }
```

```
    }
   ],
   totalTracks: 12,
   categories: ["DSA", "ML", "Development", "System_Design"],
   difficulties: ["Beginner", "Intermediate", "Advanced"],
   filters: {
    appliedFilters: {
     category: "DSA",
     difficulty: "Intermediate",
     year: 2
    }
   }
  }
}
```

GET /api/tracks/:trackId

```
// URL: /api/tracks/dsa_intermediate
// Response
{
  success: true,
  data: {
    track: {
      trackId: "dsa_intermediate",
      title: "Data Structures & Algorithms - Intermediate",
      description: "Master intermediate DSA concepts with hands-on practice",
      category: "DSA",
      difficulty: "Intermediate",
      estimatedDuration: 45,

      userProgress: {
        isEnrolled: true,
        enrollmentDate: "2024-08-01T00:00:00Z",
        overallProgress: 65,
        currentTopic: "sliding_window",
        totalTimeSpent: 1280, // minutes
        certificateEligible: false
      },

      topics: [
        {
          topicId: "arrays_advanced",
          title: "Advanced Array Techniques",
          description: "Master advanced array manipulation techniques",
```

```
  order: 1,
  estimatedTime: 8,
  difficulty: "Medium",
  isCompleted: true,
  completionDate: "2024-08-10T00:00:00Z",

  subtopics: [
    {
      subtopicId: "two_pointers",
      title: "Two Pointers Technique",
      concepts: ["Left-Right Pointers", "Fast-Slow Pointers", "Sliding Window Variation"],
      isCompleted: true,
      problemsCount: 8,
      solvedProblems: 8,
      accuracy: 87.5
    },
    {
      subtopicId: "prefix_sum",
      title: "Prefix Sum Arrays",
      concepts: ["Cumulative Sum", "Range Queries", "2D Prefix Sum"],
      isCompleted: true,
      problemsCount: 6,
      solvedProblems: 6,
      accuracy: 91.2
    }
  ],

  totalProblems: 14,
  solvedProblems: 14,
  averageAccuracy: 89.1,
  quizScore: 92,
  isLocked: false
},

{
  topicId: "sliding_window",
  title: "Sliding Window Technique",
  description: "Optimize array problems using sliding window approach",
  order: 2,
  estimatedTime: 6,
  difficulty: "Medium",
  isCompleted: false,

  subtopics: [
```

```
    {
      subtopicId: "fixed_window",
      title: "Fixed Size Window",
      concepts: ["Maximum Sum Subarray", "Average Calculation", "Window Operations"],
      isCompleted: true,
      problemsCount: 5,
      solvedProblems: 5,
      accuracy: 80.0
    },
    {
      subtopicId: "variable_window",
      title: "Variable Size Window",
      concepts: ["Longest Substring", "Minimum Window", "Optimal Subarray"],
      isCompleted: false,
      problemsCount: 7,
      solvedProblems: 3,
      accuracy: 66.7,
      currentProblem: "Longest Substring Without Repeating Characters",
      lastAttempted: "2024-08-25T11:30:00Z"
    }
  ],

  totalProblems: 12,
  solvedProblems: 8,
  averageAccuracy: 72.5,
  quizScore: null,
  isLocked: false,
  progress: 67
},

{
  topicId: "hash_maps",
  title: "Hash Map Patterns",
  description: "Leverage hash maps for efficient problem solving",
  order: 3,
  estimatedTime: 7,
  difficulty: "Medium",
  isCompleted: false,

  subtopics: [
    {
      subtopicId: "frequency_counting",
      title: "Frequency Counting",
      concepts: ["Character Frequency", "Element Counting", "Anagram Detection"],
```

```
        isCompleted: false,
        problemsCount: 6,
        solvedProblems: 0,
        accuracy: 0
      }
    ],

    totalProblems: 15,
    solvedProblems: 0,
    averageAccuracy: 0,
    quizScore: null,
    isLocked: false,
    unlockCriteria: {
      prerequisiteTopics: ["sliding_window"],
      minAccuracy: 75,
      minProblems: 10
    }
  }
],

overallStats: {
  totalTopics: 8,
  completedTopics: 1,
  totalProblems: 120,
  solvedProblems: 22,
  overallAccuracy: 81.2,
  totalTimeSpent: 1280,
  averageTimePerProblem: 58,
  strongAreas: ["Two Pointers", "Prefix Sum"],
  improvementAreas: ["Variable Window", "Hash Map Patterns"]
},

recommendations: {
  nextTopic: "hash_maps",
  suggestedProblems: [
    {
      problemId: "two_sum",
      title: "Two Sum",
      difficulty: "Easy",
      reason: "Foundation for hash map concepts"
    }
  ],
  studyPlan: {
    dailyTime: 90,
```

```
        problemsPerDay: 3,
        estimatedCompletion: "2024-09-15T00:00:00Z"
      }
    }
  }
 }
}
```

POST /api/tracks/:trackId/enroll

```
// URL: /api/tracks/dsa_intermediate/enroll
// Payload
{
 studySchedule: {
  dailyTime: 90, // minutes
  problemsPerDay: 3,
  preferredStudyTime: "evening"
 },
 goals: {
  targetCompletionDate: "2024-09-30T00:00:00Z",
  targetAccuracy: 85
 }
}

// Response
{
 success: true,
 message: "Successfully enrolled in DSA Intermediate track",
 data: {
  enrollmentId: "64a7b8c9d1234567890abcdf",
  trackId: "dsa_intermediate",
  enrollmentDate: "2024-08-25T12:00:00Z",
  estimatedCompletionDate: "2024-09-30T00:00:00Z",
  firstTopic: "arrays_advanced",
  creditsRequired: 0,
  creditsDeducted: 0
 }
}
```

GET /api/tracks/:trackId/topics/:topicId/learn

```
// URL: /api/tracks/dsa_intermediate/topics/sliding_window/learn
// Response
{
```

```
  success: true,
  data: {
    topic: {
      topicId: "sliding_window",
      title: "Sliding Window Technique",
      description: "Master the sliding window technique for array optimization",

      currentSubtopic: {
        subtopicId: "variable_window",
        title: "Variable Size Window",

        // AI Learning Content
        aiContent: {
          introduction: "Let's explore variable size sliding window technique. Based on your
progress with fixed windows, you're ready for this next challenge!",

          concepts: [
            {
              conceptId: "expanding_window",
              title: "Expanding the Window",
              explanation: "Learn when and how to expand the window size",
              examples: ["Longest Substring problems", "Sum-based problems"],
              keyPoints: [
                "Expand when condition is not met",
                "Track window state continuously",
                "Update result when valid window found"
              ]
            },
            {
              conceptId: "shrinking_window",
              title: "Shrinking the Window",
              explanation: "Master the art of contracting the window",
              examples: ["Minimum window problems", "Constraint satisfaction"],
              keyPoints: [
                "Shrink when condition is violated",
                "Maintain window validity",
                "Optimize for minimum/maximum criteria"
              ]
            }
          ],

          commonPatterns: [
            {
              pattern: "Longest Valid Substring",
```

```
      template: "expand until invalid, then shrink until valid",
      pseudocode: `
        left = 0
        for right in range(n):
            # Add right element to window
            while window_invalid():
                # Remove left element and shrink
                left++
            # Update result with current window
      `
    }
  ],

  interactiveExamples: [
    {
      problem: "Find longest substring without repeating characters",
      walkthrough: [
        "Start with empty window",
        "Expand by adding characters",
        "When duplicate found, shrink from left",
        "Track maximum length seen"
      ],
      visualization: "https://example.com/sliding-window-viz"
    }
  ]
},

// Resources
resources: [
  {
    type: "video",
    title: "Sliding Window Patterns Explained",
    url: "https://youtube.com/watch?v=example1",
    duration: 15,
    difficulty: "Medium",
    isRecommended: true,
    thumbnailUrl: "https://example.com/video-thumb1.jpg"
  },
  {
    type: "article",
    title: "Master Sliding Window Technique",
    url: "https://example.com/sliding-window-article",
    readTime: 8,
    difficulty: "Medium",
```

```
      isRecommended: true
    },
    {
      type: "interactive",
      title: "Sliding Window Visualizer",
      url: "https://example.com/interactive-viz",
      description: "Interactive tool to visualize sliding window",
      isRecommended: false
    }
  ],

  // Problems for practice
  problems: [
    {
      problemId: "longest_substring_without_repeating",
      title: "Longest Substring Without Repeating Characters",
      difficulty: "Medium",
      estimatedTime: 25,
      isRecommended: true,
      companies: ["Amazon", "Microsoft", "Google"],
      status: "attempted", // "not_attempted", "attempted", "solved"
      userStats: {
        attempts: 2,
        bestTime: 35,
        lastAttempted: "2024-08-25T11:30:00Z"
      }
    },
    {
      problemId: "minimum_window_substring",
      title: "Minimum Window Substring",
      difficulty: "Hard",
      estimatedTime: 35,
      isRecommended: false,
      companies: ["Facebook", "Google"],
      status: "not_attempted",
      unlockCriteria: {
        prerequisiteProblems: ["longest_substring_without_repeating"],
        minAccuracy: 80
      }
    }
  ]
},

userProgress: {
```

```
      currentSubtopicProgress: 45,
      timeSpentToday: 25,
      problemsAttemptedToday: 1,
      currentStreak: 3,
      lastStudySession: "2024-08-25T11:30:00Z"
    },

    navigationInfo: {
      previousTopic: {
        topicId: "arrays_advanced",
        title: "Advanced Array Techniques"
      },
      nextTopic: {
        topicId: "hash_maps",
        title: "Hash Map Patterns"
      },
      progressInTrack: 65
    }
  }
 }
}
```

4. Profile Page APIs

GET /api/profile/:userId

```
// URL: /api/profile/64a7b8c9d1234567890abcde
// Response
{
  success: true,
  data: {
    profile: {
      userId: "64a7b8c9d1234567890abcde",

      basicInfo: {
        firstName: "John",
        lastName: "Doe",
        rollNumber: "21CSE001",
        year: 2,
        branch: "CSE",
        college: "Your College Name",
        email: "john.doe@student.college.edu",
        phoneNumber: "+91-9876543210",
        avatar: "https://example.com/avatars/john-doe.jpg",
```

```
  bio: "Passionate about algorithms and machine learning. Aspiring software engineer.",

  socialLinks: {
    linkedinUrl: "https://linkedin.com/in/johndoe",
    githubUrl: "https://github.com/johndoe",
    portfolioUrl: "https://johndoe.dev"
  },

  careerGoals: {
    targetCompanies: ["Google", "Microsoft", "Amazon", "Netflix"],
    preferredRole: "Software Development Engineer",
    specializations: ["Backend Development", "System Design", "Machine Learning"]
  }
},

gamificationStats: {
  currentLevel: "Intermediate",
  nextLevel: "Advanced",
  progressToNextLevel: 65,

  streakInfo: {
    currentStreak: 15,
    longestStreak: 28,
    streakStartDate: "2024-08-10T00:00:00Z",
    lastActiveDate: "2024-08-25T11:30:00Z"
  },

  credits: {
    totalEarned: 2150,
    totalSpent: 350,
    currentBalance: 1800,
    tier: "Gold",
    nextTier: "Platinum",
    creditsToNextTier: 1200
  },

  rankings: {
    globalRank: 1247,
    globalPercentile: 2.1,
    collegeRank: 12,
    collegePercentile: 5.2,
    yearRank: 3,
    branchRank: 2,
```

```
    rankHistory: [
      { date: "2024-08-01", globalRank: 2156, collegeRank: 21 },
      { date: "2024-08-08", globalRank: 1874, collegeRank: 18 },
      { date: "2024-08-15", globalRank: 1523, collegeRank: 15 },
      { date: "2024-08-22", globalRank: 1247, collegeRank: 12 }
    ]
  }
},

problemSolvingStats: {
  totalProblems: {
    attempted: 312,
    solved: 245,
    accuracyRate: 78.5
  },

  difficultyBreakdown: {
    easy: {
      attempted: 145,
      solved: 120,
      accuracy: 82.8
    },
    medium: {
      attempted: 132,
      solved: 95,
      accuracy: 72.0
    },
    hard: {
      attempted: 35,
      solved: 30,
      accuracy: 85.7
    }
  },

  topicWiseStats: [
    {
      topic: "Arrays",
      problemsAttempted: 45,
      problemsSolved: 42,
      accuracy: 93.3,
      averageTime: 18,
      rank: "Expert"
    },
    {
```

```
    topic: "Dynamic Programming",
    problemsAttempted: 32,
    problemsSolved: 21,
    accuracy: 65.6,
    averageTime: 42,
    rank: "Intermediate"
  },
  {
    topic: "Graphs",
    problemsAttempted: 28,
    problemsSolved: 25,
    accuracy: 89.3,
    averageTime: 35,
    rank: "Advanced"
  }
],

languageStats: {
  mostUsed: "Python",
  distribution: {
    python: 45,
    cpp: 35,
    java: 15,
    javascript: 5
  },
  averageTimeByLanguage: {
    python: 22,
    cpp: 28,
    java: 31,
    javascript: 25
  }
},

contestPerformance: {
  contestsParticipated: 12,
  averageRank: 145,
  bestRank: 23,
  ratingHistory: [
    { date: "2024-07-01", rating: 1200, rank: 234 },
    { date: "2024-07-15", rating: 1285, rank: 187 },
    { date: "2024-08-01", rating: 1342, rank: 156 },
    { date: "2024-08-15", rating: 1398, rank: 145 },
    { date: "2024-08-25", rating: 1445, rank: 123 }
  ],
```

```
    upcomingContests: [
      {
        contestId: "weekly_dsa_challenge_34",
        title: "Weekly DSA Challenge #34",
        startDate: "2024-08-27T10:00:00Z",
        isRegistered: true
      }
    ]
  },

  studyPatterns: {
    totalStudyHours: 284,
    averageSessionDuration: 47, // minutes
    mostProductiveTime: "14:00-16:00",
    studyStreakData: [
      { date: "2024-08-19", problems: 2, hours: 1.5 },
      { date: "2024-08-20", problems: 1, hours: 0.8 },
      { date: "2024-08-21", problems: 3, hours: 2.2 },
      { date: "2024-08-22", problems: 2, hours: 1.3 },
      { date: "2024-08-23", problems: 2, hours: 1.7 },
      { date: "2024-08-24", problems: 1, hours: 1.0 },
      { date: "2024-08-25", problems: 1, hours: 0.5 }
    ],
    weeklyGoals: {
      currentWeek: {
        problemsGoal: 15,
        problemsSolved: 12,
        studyHoursGoal: 12,
        studyHoursCompleted: 8.0,
        weekProgress: 80
      }
    }
  }
},

achievements: [
  {
    achievementId: "array_master",
    title: "Array Master",
    description: "Solved 50 array problems with 90%+ accuracy",
    badgeIcon: "https://example.com/badges/array-master.svg",
    category: "Problem Solving",
    rarity: "Rare",
```

```
    unlockedAt: "2024-08-20T10:30:00Z",
    creditsEarned: 100,
    isDisplayed: true
  },
  {
    achievementId: "consistent_coder",
    title: "Consistent Coder",
    description: "Maintained 15-day coding streak",
    badgeIcon: "https://example.com/badges/consistent-coder.svg",
    category: "Consistency",
    rarity: "Epic",
    unlockedAt: "2024-08-25T12:00:00Z",
    creditsEarned: 150,
    isDisplayed: true
  },
  {
    achievementId: "contest_warrior",
    title: "Contest Warrior",
    description: "Achieved top 50 rank in college contest",
    badgeIcon: "https://example.com/badges/contest-warrior.svg",
    category: "Competition",
    rarity: "Legendary",
    unlockedAt: "2024-08-15T18:00:00Z",
    creditsEarned: 250,
    isDisplayed: true
  },
  {
    achievementId: "mentor_helper",
    title: "Helpful Mentor",
    description: "Helped 5 junior students with concepts",
    badgeIcon: "https://example.com/badges/mentor-helper.svg",
    category: "Community",
    rarity: "Common",
    unlockedAt: "2024-08-22T14:20:00Z",
    creditsEarned: 75,
    isDisplayed: false
  }
],

learningJourney: {
  tracksEnrolled: [
    {
      trackId: "dsa_intermediate",
      title: "DSA - Intermediate",
```

```
      enrollmentDate: "2024-08-01T00:00:00Z",
      progress: 65,
      status: "in_progress",
      estimatedCompletion: "2024-09-15T00:00:00Z"
    },
    {
      trackId: "system_design_basics",
      title: "System Design Basics",
      enrollmentDate: "2024-08-10T00:00:00Z",
      progress: 25,
      status: "in_progress",
      estimatedCompletion: "2024-10-01T00:00:00Z"
    }
  ],

  tracksCompleted: [
    {
      trackId: "dsa_beginner",
      title: "DSA - Beginner",
      completionDate: "2024-07-30T00:00:00Z",
      finalScore: 92,
      certificateUrl: "https://example.com/certificates/john-doe-dsa-beginner.pdf",
      creditsEarned: 200
    }
  ],

  milestones: [
    {
      milestoneId: "first_100_problems",
      title: "Century Club",
      description: "Solved first 100 problems",
      achievedAt: "2024-08-15T00:00:00Z",
      creditsEarned: 100
    },
    {
      milestoneId: "first_contest_participation",
      title: "Contest Debut",
      description: "Participated in first coding contest",
      achievedAt: "2024-07-20T00:00:00Z",
      creditsEarned: 50
    }
  ]
},
```

```
recentActivity: [
  {
    activityId: "64a7b8c9d1234567890abc01",
    type: "problem_solved",
    title: "Solved 'Longest Palindromic Substring'",
    description: "Medium difficulty problem solved in 45 minutes",
    timestamp: "2024-08-25T11:30:00Z",
    metadata: {
      problemId: "longest_palindromic_substring",
      difficulty: "Medium",
      timeTaken: 45,
      language: "Python",
      accuracy: "First Try",
      creditsEarned: 15
    }
  },
  {
    activityId: "64a7b8c9d1234567890abc02",
    type: "session_attended",
    title: "Attended 'System Design Basics' session",
    description: "90-minute session on scalable system architecture",
    timestamp: "2024-08-24T16:00:00Z",
    metadata: {
      sessionId: "system_design_basics_Aug24",
      duration: 90,
      mentor: "Priya Sharma (SDE-3 @ Amazon)",
      rating: 5,
      creditsSpent: 25
    }
  },
  {
    activityId: "64a7b8c9d1234567890abc03",
    type: "helped_junior",
    title: "Helped junior with 'Binary Trees' concept",
    description: "Explained tree traversal algorithms to 1st year student",
    timestamp: "2024-08-24T14:20:00Z",
    metadata: {
      helpedUser: "Rahul Singh (1st Year)",
      topic: "Binary Trees",
      duration: 30,
      creditsEarned: 10
    }
  },
  {
```

```
        activityId: "64a7b8c9d1234567890abc04",
        type: "contest_participated",
        title: "Participated in 'Weekly Algorithm Challenge'",
        description: "Ranked 67th out of 234 participants",
        timestamp: "2024-08-23T10:00:00Z",
        metadata: {
          contestId: "weekly_algo_challenge_33",
          rank: 67,
          totalParticipants: 234,
          problemsSolved: 2,
          totalProblems: 3,
          creditsEarned: 30
        }
      }
    ],

    socialStats: {
      followersCount: 23,
      followingCount: 18,
      postsCount: 12,
      likesReceived: 145,
      commentsGiven: 67,

      mentorshipStats: {
        juniorsHelped: 8,
        sessionsCompleted: 3,
        averageRating: 4.8,
        totalHoursSpent: 12
      }
    }
  }
 }
}
```

PUT /api/profile/update

```
// Payload
{
  profile: {
    bio: "Updated bio - Passionate about algorithms and system design",
    phoneNumber: "+91-9876543211",
    linkedinUrl: "https://linkedin.com/in/johndoe-updated",
    githubUrl: "https://github.com/johndoe-new",
    portfolioUrl: "https://johnsmith.dev"
```

```
  },
  careerGoals: {
    targetCompanies: ["Google", "Microsoft", "Amazon", "Netflix", "Meta"],
    preferredRole: "Senior Software Development Engineer",
    specializations: ["Backend Development", "System Design", "Machine Learning", "Cloud
Architecture"]
  },
  preferences: {
    preferredLanguage: "python",
    dailyGoal: 4,
    studyReminders: true,
    emailNotifications: false
  }
}

// Response
{
  success: true,
  message: "Profile updated successfully",
  data: {
    updatedFields: ["bio", "phoneNumber", "linkedinUrl", "githubUrl", "portfolioUrl",
"targetCompanies", "specializations", "dailyGoal", "emailNotifications"],
    updatedAt: "2024-08-25T12:30:00Z"
  }
}
```

GET /api/profile/:userId/achievements

```
// URL: /api/profile/64a7b8c9d1234567890abcde/achievements
// Response
{
  success: true,
  data: {
    achievements: [
      {
        achievementId: "array_master",
        title: "Array Master",
        description: "Solved 50 array problems with 90%+ accuracy",
        badgeIcon: "https://example.com/badges/array-master.svg",
        category: "Problem Solving",
        rarity: "Rare",
        rarityColor: "#9333ea",
        unlockedAt: "2024-08-20T10:30:00Z",
        creditsEarned: 100,
```

```
    progress: 100,
    isDisplayed: true,

    requirements: {
      description: "Solve 50 array problems with minimum 90% accuracy",
      criteria: [
        { type: "problems_solved", topic: "Arrays", target: 50, current: 50 },
        { type: "accuracy", topic: "Arrays", target: 90, current: 93.3 }
      ]
    }
  },

  // Progress achievements (not yet unlocked)
  {
    achievementId: "dp_master",
    title: "Dynamic Programming Master",
    description: "Solve 30 DP problems with 85%+ accuracy",
    badgeIcon: "https://example.com/badges/dp-master.svg",
    category: "Problem Solving",
    rarity: "Epic",
    rarityColor: "#dc2626",
    isUnlocked: false,
    progress: 70,

    requirements: {
      description: "Solve 30 DP problems with minimum 85% accuracy",
      criteria: [
        { type: "problems_solved", topic: "Dynamic Programming", target: 30, current: 21 },
        { type: "accuracy", topic: "Dynamic Programming", target: 85, current: 65.6 }
      ]
    }
  }
],

categoryStats: {
  "Problem Solving": { unlocked: 3, total: 8 },
  "Consistency": { unlocked: 2, total: 5 },
  "Competition": { unlocked: 1, total: 6 },
  "Community": { unlocked: 1, total: 4 },
  "Learning": { unlocked: 2, total: 7 }
},

totalCreditsEarned: 685,
rarityBreakdown: {
```

```
    "Common": 3,
    "Rare": 2,
    "Epic": 2,
    "Legendary": 1
   }
  }
}
```

5. Problem & Code Execution APIs

GET /api/problems/:problemId

```
// URL: /api/problems/two_sum
// Response
{
  success: true,
  data: {
    problem: {
      problemId: "two_sum",
      title: "Two Sum",
      description: "Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target.\n\nYou may assume that each input would have exactly one solution, and you may not use the same element twice.\n\nYou can return the answer in any order.",
      difficulty: "Easy",
      topics: ["Array", "Hash Table"],
      subtopics: ["Two Pointers", "Hash Map"],
      companies: ["Amazon", "Google", "Microsoft", "Apple", "Facebook"],
      frequency: 9.2,

      constraints: {
        timeLimit: 1000, // milliseconds
        memoryLimit: 256, // MB
        inputConstraints: [
          "2 <= nums.length <= 10^4",
          "-10^9 <= nums[i] <= 10^9",
          "-10^9 <= target <= 10^9",
          "Only one valid answer exists"
        ]
      },

      examples: [
        {
          input: "nums = [2,7,11,15], target = 9",
```

```
      output: "[0,1]",
      explanation: "Because nums[0] + nums[1] == 9, we return [0, 1]."
    },
    {
      input: "nums = [3,2,4], target = 6",
      output: "[1,2]",
      explanation: "nums[1] + nums[2] == 6, so we return [1, 2]."
    },
    {
      input: "nums = [3,3], target = 6",
      output: "[0,1]",
      explanation: "nums[0] + nums[1] == 6, so we return [0, 1]."
    }
  ],

  hints: [
    "A really brute force way would be to search for all possible pairs of numbers but that would be too slow. Again, it's best to try out brute force solutions for such problems if you have no idea.",
    "So, if we fix one of the numbers, say x, we have to scan the entire array to find the next number y which is value - x where value is the input parameter. Can we change our array somehow so that this search becomes faster?",
    "The second train of thought is, without changing the array, can we use additional space somehow? Like maybe a hash map to speed up the search?"
  ],

  userStats: {
    totalAttempts: 15432,
    acceptedSubmissions: 12891,
    acceptanceRate: 83.5,
    averageTimeToSolve: 18, // minutes

    userProgress: {
      attempts: 3,
      isAccepted: true,
      bestSubmission: {
        submissionId: "64a7b8c9d1234567890abce1",
        submittedAt: "2024-08-20T10:15:00Z",
        language: "python",
        runtime: 52, // ms
        memory: 15.1, // MB
        status: "Accepted"
      },
      firstAcceptedAt: "2024-08-20T10:15:00Z",
```

```
        totalTimeSpent: 25,
        bookmarked: true,
        notes: "Use hash map for O(1) lookup. Remember to check if complement exists before adding current number to map."
      }
    },

    solutions: [
      {
        approach: "Brute Force",
        timeComplexity: "O(n²)",
        spaceComplexity: "O(1)",
        description: "Check every pair of numbers",
        isOptimal: false
      },
      {
        approach: "Hash Map",
        timeComplexity: "O(n)",
        spaceComplexity: "O(n)",
        description: "Use hash map to store complements",
        isOptimal: true
      }
    ],

    relatedProblems: [
      {
        problemId: "3sum",
        title: "3Sum",
        difficulty: "Medium",
        similarity: 85
      },
      {
        problemId: "two_sum_ii",
        title: "Two Sum II - Input Array Is Sorted",
        difficulty: "Easy",
        similarity: 92
      }
    ]
  }
 }
}
```

POST /api/problems/:problemId/submit

```
// URL: /api/problems/two_sum/submit
// Payload
{
  code: `def twoSum(nums, target):
    hash_map = {}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in hash_map:
            return [hash_map[complement], i]
        hash_map[num] = i
    return []`,
  language: "python",
  testMode: false // false for submission, true for testing
}

// Response
{
  success: true,
  data: {
    submissionId: "64a7b8c9d1234567890abce2",
    status: "Accepted", // "Accepted", "Wrong Answer", "Time Limit Exceeded", "Runtime Error",
"Compilation Error"

    executionResults: {
      totalTestCases: 57,
      passedTestCases: 57,
      failedTestCases: 0,

      publicTestResults: [
        {
          testCaseIndex: 1,
          status: "Passed",
          executionTime: 45,
          memoryUsed: 15.2,
          input: "[2,7,11,15]\n9",
          expectedOutput: "[0,1]",
          actualOutput: "[0,1]"
        },
        {
          testCaseIndex: 2,
          status: "Passed",
          executionTime: 42,
          memoryUsed: 15.1,
          input: "[3,2,4]\n6",
```

```
        expectedOutput: "[1,2]",
        actualOutput: "[1,2]"
      }
    ],

    overallStats: {
      totalRuntime: 52,
      memoryUsage: 15.3,
      runtimePercentile: 78.5,
      memoryPercentile: 82.1
    }
  },

  submissionMetrics: {
    timeTakenToSolve: 22, // minutes since problem opened
    isFirstAccepted: false,
    attemptNumber: 3,
    codeQualityScore: 8.5,

    creditsEarned: 15,
    experiencePoints: 25,

    achievements: [
      {
        achievementId: "problem_solver",
        title: "Problem Solver",
        description: "Solved your 25th problem",
        creditsEarned: 10
      }
    ]
  },

  feedback: {
    aiGeneratedFeedback: "Excellent solution! You used the optimal hash map approach with
O(n) time complexity. Your code is clean and well-structured.",
    performanceAnalysis: "Your solution performed better than 78.5% of submissions in runtime
and 82.1% in memory usage.",
    nextSteps: [
      "Try the follow-up: 3Sum problem",
      "Practice more hash map problems",
      "Learn about space-time trade-offs"
    ]
  }
}
```

```
}
```

POST /api/problems/:problemId/run-tests

```
// URL: /api/problems/two_sum/run-tests
// Payload
{
  code: `def twoSum(nums, target):
    hash_map = {}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in hash_map:
            return [hash_map[complement], i]
        hash_map[num] = i
    return []`,
  language: "python"
}

// Response
{
  success: true,
  data: {
    testResults: [
      {
        testCaseIndex: 1,
        status: "Passed",
        executionTime: 45,
        memoryUsed: 15.2,
        input: "[2,7,11,15]\n9",
        expectedOutput: "[0,1]",
        actualOutput: "[0,1]"
      },
      {
        testCaseIndex: 2,
        status: "Passed",
        executionTime: 42,
        memoryUsed: 15.1,
        input: "[3,2,4]\n6",
        expectedOutput: "[1,2]",
        actualOutput: "[1,2]"
      },
      {
        testCaseIndex: 3,
        status: "Failed",
```

```
      executionTime: 38,
      memoryUsed: 14.9,
      input: "[3,3]\n6",
      expectedOutput: "[0,1]",
      actualOutput: "[1,0]",
      errorMessage: "Output order doesn't match expected result"
    }
   ],

   summary: {
    totalTests: 3,
    passedTests: 2,
    failedTests: 1,
    overallStatus: "Some tests failed"
   }
  }
}
```

Frontend Pages & Components

1. Home Page (/pages/index.js)

Required Data from Backend

```
// API Calls needed on page load
const homePageData = {
  // From GET /api/dashboard/home
  userProgress: { /* sprint info, streak, credits */ },
  notifications: [ /* upcoming events */ ],
  leaderboard: { /* weekly rankings */ },
  quickActions: [ /* continue learning, tests, etc */ ],
  recentActivity: [ /* recent achievements */ ],

  // From GET /api/dashboard/weekly-stats
  weeklyStats: { /* study patterns, progress trends */ }
}
```

Main Components

```
// components/home/ProgressOverviewCard.js
const ProgressOverviewCard = ({ userProgress }) => {
  // Display current sprint, streak, credits
  // Show progress bar for current track
```

```
  // Quick stats like problems solved today
}

// components/home/NotificationsPanel.js
const NotificationsPanel = ({ notifications }) => {
  // Scrollable list of upcoming events
  // Action buttons for registration
  // Priority highlighting for important events
}

// components/home/LeaderboardWidget.js
const LeaderboardWidget = ({ leaderboard }) => {
  // Top 3 rankers with user position
  // Quick view toggle (weekly/monthly/overall)
  // Link to full leaderboard page
}

// components/home/QuickActionsGrid.js
const QuickActionsGrid = ({ actions }) => {
  // 2x2 grid of primary actions
  // Dynamic content based on user progress
  // Visual indicators for recommended actions
}

// components/home/ActivityFeed.js
const ActivityFeed = ({ recentActivity }) => {
  // Timeline view of recent actions
  // Achievement highlights
  // Social interactions (helps, comments)
}
```

Page Structure

```
// pages/index.js
import { useState, useEffect } from 'react';
import { useAuth } from '@/hooks/useAuth';

export default function HomePage() {
  const { user } = useAuth();
  const [homeData, setHomeData] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchHomeData = async () => {
```

```
    try {
      // Parallel API calls
      const [dashboardRes, weeklyStatsRes] = await Promise.all([
        fetch('/api/dashboard/home', {
          headers: { Authorization: `Bearer ${user.token}` }
        }),
        fetch('/api/dashboard/weekly-stats', {
          headers: { Authorization: `Bearer ${user.token}` }
        })
      ]);

      const dashboardData = await dashboardRes.json();
      const weeklyData = await weeklyStatsRes.json();

      setHomeData({
        ...dashboardData.data,
        weeklyStats: weeklyData.data
      });
    } catch (error) {
      console.error('Error fetching home data:', error);
    } finally {
      setLoading(false);
    }
  };

  if (user) {
    fetchHomeData();
  }
}, [user]);

if (loading) return <HomePageSkeleton />;

return (
  <div className="min-h-screen bg-gray-50 p-6">
    <div className="max-w-7xl mx-auto space-y-6">
      {/* Main Progress Section */}
      <div className="grid grid-cols-1 lg:grid-cols-3 gap-6">
        <div className="lg:col-span-2">
          <ProgressOverviewCard
            userProgress={homeData.userProgress}
            weeklyStats={homeData.weeklyStats}
          />
        </div>
        <div>
```

```
          <LeaderboardWidget leaderboard={homeData.leaderboard} />
        </div>
      </div>

      {/* Notifications & Quick Actions */}
      <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
        <NotificationsPanel notifications={homeData.notifications} />
        <QuickActionsGrid actions={homeData.quickActions} />
      </div>

      {/* Activity Feed */}
      <ActivityFeed recentActivity={homeData.recentActivity} />
    </div>
  </div>
 );
}
```

2. Tracks Page (/pages/tracks/index.js)

Required Data from Backend

```
// API Calls needed
const tracksPageData = {
 // From GET /api/tracks with filters
 tracks: [ /* all available tracks */ ],
 categories: [ /* DSA, ML, Development */ ],
 userEnrollments: [ /* user's enrolled tracks */ ],

 // From GET /api/tracks/recommendations
 recommendedTracks: [ /* personalized recommendations */ ]
}
```

Main Components

```
// components/tracks/TracksFilter.js
const TracksFilter = ({ onFilterChange, categories, difficulties }) => {
 // Category tabs (DSA, ML, Development)
 // Difficulty selector
 // Year filter (1st, 2nd, 3rd, 4th)
 // Search functionality
}

// components/tracks/TrackCard.js
const TrackCard = ({ track, userProgress }) => {
```

```
  // Track thumbnail and basic info
  // Progress bar if enrolled
  // Enrollment/Continue button
  // Prerequisites display
  // Estimated duration and difficulty
}

// components/tracks/EnrolledTracksSection.js
const EnrolledTracksSection = ({ enrolledTracks }) => {
  // Horizontal scrollable cards
  // Quick access to continue learning
  // Progress indicators
}
```

Page Structure

```
// pages/tracks/index.js
export default function TracksPage() {
  const [tracks, setTracks] = useState([]);
  const [filters, setFilters] = useState({
    category: 'all',
    difficulty: 'all',
    year: 'all',
    search: ''
  });
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchTracks = async () => {
      try {
        const queryParams = new URLSearchParams();
        if (filters.category !== 'all') queryParams.append('category', filters.category);
        if (filters.difficulty !== 'all') queryParams.append('difficulty', filters.difficulty);
        if (filters.year !== 'all') queryParams.append('year', filters.year);
        if (filters.search) queryParams.append('search', filters.search);

        const response = await fetch(`/api/tracks?${queryParams.toString()}`, {
          headers: { Authorization: `Bearer ${user.token}` }
        });

        const data = await response.json();
        setTracks(data.data.tracks);
      } catch (error) {
        console.error('Error fetching tracks:', error);
```

```
      } finally {
        setLoading(false);
      }
    };

    fetchTracks();
  }, [filters]);

  return (
    <div className="min-h-screen bg-gray-50 p-6">
      <div className="max-w-7xl mx-auto">
        {/* Header */}
        <div className="mb-8">
          <h1 className="text-3xl font-bold text-gray-900">Learning Tracks</h1>
          <p className="text-gray-600 mt-2">Choose your learning path and start your
journey</p>
        </div>

        {/* Enrolled Tracks Section */}
        <EnrolledTracksSection enrolledTracks={enrolledTracks} />

        {/* Filters */}
        <TracksFilter
          onFilterChange={setFilters}
          currentFilters={filters}
          categories={categories}
          difficulties={difficulties}
        />

        {/* Tracks Grid */}
        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6 mt-8">
          {loading ? (
            Array(6).fill(0).map((_, index) => <TrackCardSkeleton key={index} />)
          ) : (
            tracks.map(track => (
              <TrackCard
                key={track.trackId}
                track={track}
                userProgress={track.userProgress}
              />
            ))
          )}
        </div>
      </div>
```

```
    </div>
  );
}
```

3. Individual Track Learning Page (/pages/tracks/[trackId]/learn/[topicId].js)

Required Data from Backend

```
// API Call needed
const learningPageData = {
  // From GET /api/tracks/:trackId/topics/:topicId/learn
  topic: { /* AI content, resources, problems */ },
  userProgress: { /* current progress, time spent */ },
  navigationInfo: { /* previous/next topics */ }
}
```

Main Components

```
// components/learning/AITutorChat.js
const AITutorChat = ({ topicId, aiContent, userProgress }) => {
  // Chat interface for AI learning
  // Conversational problem explanations
  // Adaptive content based on user level
  // Progress tracking within chat
}

// components/learning/ResourcesPanel.js
const ResourcesPanel = ({ resources }) => {
  // Collapsible video/article list
  // Resource filtering by type
  // Progress tracking for consumption
}

// components/learning/CodeEditor.js
const CodeEditor = ({ problem, onSubmission }) => {
  // Monaco editor integration
  // Language selection
  // Test case runner
  // Submission handler
}

// components/learning/ProblemsList.js
const ProblemsList = ({ problems, onProblemSelect }) => {
  // Filterable problem list
```

```
  // Difficulty indicators
  // Company tags
  // Solution status
}
```

Page Structure

```
// pages/tracks/[trackId]/learn/[topicId].js
export default function TopicLearningPage() {
  const router = useRouter();
  const { trackId, topicId } = router.query;
  const [learningData, setLearningData] = useState(null);
  const [selectedProblem, setSelectedProblem] = useState(null);
  const [learningMode, setLearningMode] = useState('ai'); // 'ai', 'resources', 'practice'

  useEffect(() => {
    const fetchLearningData = async () => {
      try {
        const response = await fetch(`/api/tracks/${trackId}/topics/${topicId}/learn`, {
          headers: { Authorization: `Bearer ${user.token}` }
        });

        const data = await response.json();
        setLearningData(data.data);
      } catch (error) {
        console.error('Error fetching learning data:', error);
      }
    };

    if (trackId && topicId) {
      fetchLearningData();
    }
  }, [trackId, topicId]);

  return (
    <div className="min-h-screen bg-gray-50">
      {/* Learning Mode Tabs */}
      <div className="bg-white border-b">
        <div className="max-w-7xl mx-auto px-6">
          <div className="flex space-x-8">
            <button
              className={`py-4 px-2 border-b-2 ${learningMode === 'ai' ? 'border-blue-500' :
'border-transparent'}`}
              onClick={() => setLearningMode('ai')}
```

```
          >
            AI Tutor
          </button>
          <button
            className={`py-4 px-2 border-b-2 ${learningMode === 'resources' ? 'border-blue-500'
: 'border-transparent'}`}
            onClick={() => setLearningMode('resources')}
          >
            Resources
          </button>
          <button
            className={`py-4 px-2 border-b-2 ${learningMode === 'practice' ? 'border-blue-500' :
'border-transparent'}`}
            onClick={() => setLearningMode('practice')}
          >
            Practice
          </button>
        </div>
      </div>
    </div>

    {/* Main Learning Interface */}
    <div className="max-w-7xl mx-auto p-6">
      <div className="grid grid-cols-1 lg:grid-cols-3 gap-6 h-[calc(100vh-200px)]">
        {/* Primary Learning Area */}
        <div className="lg:col-span-2 bg-white rounded-lg shadow">
          {learningMode === 'ai' && (
            <AITutorChat
              topicId={topicId}
              aiContent={learningData?.topic?.currentSubtopic?.aiContent}
              userProgress={learningData?.topic?.userProgress}
            />
          )}
          {learningMode === 'resources' && (
            <ResourcesPanel resources={learningData?.topic?.currentSubtopic?.resources} />
          )}
          {learningMode === 'practice' && selectedProblem && (
            <CodeEditor
              problem={selectedProblem}
              onSubmission={handleProblemSubmission}
            />
          )}
        </div>
```

```
        {/* Side Panel */}
        <div className="bg-white rounded-lg shadow">
          {learningMode === 'practice' ? (
            <ProblemsList
              problems={learningData?.topic?.currentSubtopic?.problems}
              onProblemSelect={setSelectedProblem}
            />
          ) : (
            <div className="p-4">
              {/* Topic Progress */}
              <div className="mb-6">
                <h3 className="font-semibold mb-2">Topic Progress</h3>
                <div className="bg-gray-200 rounded-full h-2">
                  <div
                    className="bg-blue-500 h-2 rounded-full"
                    style={{ width: `${learningData?.topic?.userProgress?.currentSubtopicProgress ||
0}%` }}
                  />
                </div>
                <p className="text-sm text-gray-600 mt-1">
                  {learningData?.topic?.userProgress?.currentSubtopicProgress || 0}% Complete
                </p>
              </div>

              {/* Quick Problem Access */}
              <div className="mb-6">
                <h3 className="font-semibold mb-2">Quick Practice</h3>
                <button
                  className="w-full bg-blue-500 text-white py-2 rounded hover:bg-blue-600"
                  onClick={() => setLearningMode('practice')}
                >
                  Start Practicing
                </button>
              </div>

              {/* Navigation */}
              <div>
                <h3 className="font-semibold mb-2">Navigation</h3>
                <div className="space-y-2">
                  {learningData?.topic?.navigationInfo?.previousTopic && (
                    <button className="w-full text-left p-2 text-sm bg-gray-50 rounded
hover:bg-gray-100">
                      ← {learningData.topic.navigationInfo.previousTopic.title}
                    </button>
```

```
                )}
                {learningData?.topic?.navigationInfo?.nextTopic && (
                  <button className="w-full text-left p-2 text-sm bg-gray-50 rounded
hover:bg-gray-100">
                    {learningData.topic.navigationInfo.nextTopic.title} →
                  </button>
                )}
              </div>
            </div>
          </div>
        )}
      </div>
    </div>
  </div>
 );
}
```

4. Profile Page (/pages/profile/[userId].js)

Required Data from Backend

```
// API Calls needed
const profilePageData = {
  // From GET /api/profile/:userId
  profile: { /* complete profile data */ },

  // From GET /api/profile/:userId/achievements
  achievements: [ /* all achievements with progress */ ],

  // Additional data for charts and analytics
  performanceAnalytics: { /* charts data */ }
}
```

Main Components

```
// components/profile/ProfileHeader.js
const ProfileHeader = ({ profile }) => {
  // Avatar, name, basic info
  // Current streak, level, credits
  // Social links and contact info
  // Edit profile button (if own profile)
}
```

```
// components/profile/StatsOverview.js
const StatsOverview = ({ stats }) => {
  // Key metrics in card layout
  // Problems solved, accuracy, rank
  // Visual progress indicators
  // Comparison with peers
}

// components/profile/AchievementsBadges.js
const AchievementsBadges = ({ achievements }) => {
  // Grid of unlocked badges
  // Progress indicators for locked achievements
  // Rarity-based styling
  // Achievement details modal
}

// components/profile/PerformanceCharts.js
const PerformanceCharts = ({ analytics }) => {
  // Problem-solving trends
  // Topic-wise accuracy radar chart
  // Contest performance timeline
  // Study pattern heatmap
}

// components/profile/ActivityTimeline.js
const ActivityTimeline = ({ recentActivity }) => {
  // Chronological activity feed
  // Achievement highlights
  // Social interactions
  // Load more functionality
}

// components/profile/LearningJourney.js
const LearningJourney = ({ tracks, milestones }) => {
  // Track completion progress
  // Learning path visualization
  // Milestone achievements
  // Certificates earned
}
```

Page Structure

```
// pages/profile/[userId].js
export default function ProfilePage() {
```

```
const router = useRouter();
const { userId } = router.query;
const { user } = useAuth();
const [profileData, setProfileData] = useState(null);
const [activeTab, setActiveTab] = useState('overview');
const [loading, setLoading] = useState(true);

const isOwnProfile = user?.id === userId;

useEffect(() => {
  const fetchProfileData = async () => {
    try {
      const [profileRes, achievementsRes] = await Promise.all([
        fetch(`/api/profile/${userId}`, {
          headers: { Authorization: `Bearer ${user.token}` }
        }),
        fetch(`/api/profile/${userId}/achievements`, {
          headers: { Authorization: `Bearer ${user.token}` }
        })
      ]);

      const profileData = await profileRes.json();
      const achievementsData = await achievementsRes.json();

      setProfileData({
        ...profileData.data,
        achievements: achievementsData.data.achievements
      });
    } catch (error) {
      console.error('Error fetching profile data:', error);
    } finally {
      setLoading(false);
    }
  };

  if (userId && user) {
    fetchProfileData();
  }
}, [userId, user]);

if (loading) return <ProfilePageSkeleton />;

return (
  <div className="min-h-screen bg-gray-50">
```

```
      {/* Profile Header */}
      <div className="bg-white border-b">
        <div className="max-w-7xl mx-auto px-6 py-8">
          <ProfileHeader
            profile={profileData.profile}
            isOwnProfile={isOwnProfile}
          />
        </div>
      </div>

      {/* Tab Navigation */}
      <div className="bg-white border-b">
        <div className="max-w-7xl mx-auto px-6">
          <div className="flex space-x-8">
            {['overview', 'achievements', 'analytics', 'activity', 'journey'].map(tab => (
              <button
                key={tab}
                className={`py-4 px-2 border-b-2 capitalize ${
                  activeTab === tab ? 'border-blue-500 text-blue-600' : 'border-transparent
text-gray-600'
                }`}
                onClick={() => setActiveTab(tab)}
              >
                {tab}
              </button>
            ))}
          </div>
        </div>
      </div>

      {/* Tab Content */}
      <div className="max-w-7xl mx-auto px-6 py-8">
        {activeTab === 'overview' && (
          <div className="grid grid-cols-1 lg:grid-cols-3 gap-8">
            <div className="lg:col-span-2 space-y-6">
              <StatsOverview stats={profileData.profile.problemSolvingStats} />
              <PerformanceCharts analytics={profileData.profile.problemSolvingStats} />
            </div>
            <div className="space-y-6">
              <AchievementsBadges achievements={profileData.achievements.slice(0, 6)} />
              <ActivityTimeline recentActivity={profileData.profile.recentActivity.slice(0, 5)} />
            </div>
          </div>
        )}
```

```
{activeTab === 'achievements' && (
  <AchievementsBadges
    achievements={profileData.achievements}
    showAll={true}
  />
)}

{activeTab === 'analytics' && (
  <div className="grid grid-cols-1 lg:grid-cols-2 gap-8">
    <PerformanceCharts analytics={profileData.profile.problemSolvingStats} />
    <div className="space-y-6">
      {/* Topic-wise performance */}
      <div className="bg-white p-6 rounded-lg shadow">
        <h3 className="text-lg font-semibold mb-4">Topic Performance</h3>
        {profileData.profile.problemSolvingStats.topicWiseStats.map(topic => (
          <div key={topic.topic} className="mb-4">
            <div className="flex justify-between items-center mb-1">
              <span className="text-sm font-medium">{topic.topic}</span>
              <span className="text-sm text-gray-600">{topic.accuracy.toFixed(1)}%</span>
            </div>
            <div className="bg-gray-200 rounded-full h-2">
              <div
                className={`h-2 rounded-full ${
                  topic.accuracy >= 90 ? 'bg-green-500' :
                  topic.accuracy >= 75 ? 'bg-yellow-500' : 'bg-red-500'
                }`}
                style={{ width: `${topic.accuracy}%` }}
              />
            </div>
          </div>
        ))}
      </div>
    </div>
  </div>
)}

{activeTab === 'activity' && (
  <ActivityTimeline
    recentActivity={profileData.profile.recentActivity}
    showAll={true}
  />
)}
```

```
        {activeTab === 'journey' && (
          <LearningJourney
            tracks={profileData.profile.learningJourney}
            milestones={profileData.profile.learningJourney.milestones}
          />
        )}
      </div>
    </div>
  );
}
```

Middleware & Authentication

1. Auth Middleware

```
// middleware/auth.js
import jwt from 'jsonwebtoken';
import User from '@/models/User';

export const authenticateToken = async (req, res, next) => {
  try {
    const authHeader = req.headers['authorization'];
    const token = authHeader && authHeader.split(' ')[1];

    if (!token) {
      return res.status(401).json({
        success: false,
        message: 'Access token is required'
      });
    }

    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(decoded.userId).select('-password');

    if (!user) {
      return res.status(401).json({
        success: false,
        message: 'Invalid token - user not found'
      });
    }

    if (!user.isVerified) {
      return res.status(401).json({
```

```
        success: false,
        message: 'Please verify your email first'
      });
    }

    req.user = user;
    next();
  } catch (error) {
    return res.status(403).json({
      success: false,
      message: 'Invalid or expired token'
    });
  }
};

// Optional middleware for routes that work with or without auth
export const optionalAuth = async (req, res, next) => {
  try {
    const authHeader = req.headers['authorization'];
    const token = authHeader && authHeader.split(' ')[1];

    if (token) {
      const decoded = jwt.verify(token, process.env.JWT_SECRET);
      const user = await User.findById(decoded.userId).select('-password');
      req.user = user;
    }

    next();
  } catch (error) {
    // Continue without user if token is invalid
    next();
  }
};
```

2. Rate Limiting Middleware

```
// middleware/rateLimiter.js
import rateLimit from 'express-rate-limit';

export const apiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // limit each IP to 100 requests per windowMs
  message: {
    success: false,
```

```
    message: 'Too many requests, please try again later'
  },
  standardHeaders: true,
  legacyHeaders: false
});

export const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 5, // limit each IP to 5 auth requests per windowMs
  message: {
    success: false,
    message: 'Too many authentication attempts, please try again later'
  }
});

export const submissionLimiter = rateLimit({
  windowMs: 1 * 60 * 1000, // 1 minute
  max: 10, // limit each IP to 10 submissions per minute
  message: {
    success: false,
    message: 'Too many submissions, please wait before trying again'
  }
});
```

3. Validation Middleware

```
// middleware/validation.js
import { body, param, query, validationResult } from 'express-validator';

export const handleValidationErrors = (req, res, next) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({
      success: false,
      message: 'Validation failed',
      errors: errors.array()
    });
  }
  next();
};

// Validation rules
export const registerValidation = [
  body('email')
```

```
    .isEmail()
    .normalizeEmail()
    .withMessage('Please provide a valid email'),
  body('password')
    .isLength({ min: 8 })
    .matches(/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)/)
    .withMessage('Password must be at least 8 characters with uppercase, lowercase, and
number'),
  body('firstName')
    .trim()
    .isLength({ min: 2, max: 50 })
    .withMessage('First name must be between 2-50 characters'),
  body('lastName')
    .trim()
    .isLength({ min: 2, max: 50 })
    .withMessage('Last name must be between 2-50 characters'),
  body('rollNumber')
    .trim()
    .matches(/^[0-9]{2}[A-Z]{3}[0-9]{3}$/)
    .withMessage('Roll number format: 21CSE001'),
  body('year')
    .isInt({ min: 1, max: 4 })
    .withMessage('Year must be between 1-4'),
  body('branch')
    .isIn(['CSE', 'ECE', 'IT', 'EEE', 'MECH', 'CIVIL'])
    .withMessage('Invalid branch selection')
];

export const loginValidation = [
  body('email')
    .isEmail()
    .normalizeEmail()
    .withMessage('Please provide a valid email'),
  body('password')
    .notEmpty()
    .withMessage('Password is required')
];

export const profileUpdateValidation = [
  body('profile.phoneNumber')
    .optional()
    .matches(/^\+91-[0-9]{10}$/)
    .withMessage('Phone number format: +91-9876543210'),
  body('profile.bio')
```

```
    .optional()
    .isLength({ max: 500 })
    .withMessage('Bio must be less than 500 characters'),
  body('careerGoals.targetCompanies')
    .optional()
    .isArray({ max: 10 })
    .withMessage('Maximum 10 target companies allowed'),
  body('preferences.dailyGoal')
    .optional()
    .isInt({ min: 1, max: 10 })
    .withMessage('Daily goal must be between 1-10 problems')
];

export const trackParamValidation = [
  param('trackId')
    .matches(/^[a-z_]+$/)
    .withMessage('Invalid track ID format')
];

export const problemParamValidation = [
  param('problemId')
    .matches(/^[a-z_]+$/)
    .withMessage('Invalid problem ID format')
];
```

Database Utilities & Helpers

1. Database Connection

```
// lib/mongodb.js
import { MongoClient } from 'mongodb';
import mongoose from 'mongoose';

const MONGODB_URI = process.env.MONGODB_URI;

if (!MONGODB_URI) {
  throw new Error('Please define the MONGODB_URI environment variable');
}

let cached = global.mongoose;

if (!cached) {
  cached = global.mongoose = { conn: null, promise: null };
```

```
}

async function dbConnect() {
  if (cached.conn) {
    return cached.conn;
  }

  if (!cached.promise) {
    const opts = {
      bufferCommands: false,
      maxPoolSize: 10,
      serverSelectionTimeoutMS: 5000,
      socketTimeoutMS: 45000,
      bufferMaxEntries: 0,
      useNewUrlParser: true,
      useUnifiedTopology: true,
    };

    cached.promise = mongoose.connect(MONGODB_URI, opts).then((mongoose) => {
      return mongoose;
    });
  }

  try {
    cached.conn = await cached.promise;
  } catch (e) {
    cached.promise = null;
    throw e;
  }

  return cached.conn;
}

export default dbConnect;
```

2. API Response Utilities

```
// utils/apiResponse.js
export const sendSuccess = (res, data = null, message = 'Success', statusCode = 200) => {
  return res.status(statusCode).json({
    success: true,
    message,
    data,
    timestamp: new Date().toISOString()
```

```
  });
};

export const sendError = (res, message = 'Internal Server Error', statusCode = 500, errors =
null) => {
  return res.status(statusCode).json({
    success: false,
    message,
    errors,
    timestamp: new Date().toISOString()
  });
};

export const sendValidationError = (res, errors) => {
  return res.status(400).json({
    success: false,
    message: 'Validation failed',
    errors: errors.array(),
    timestamp: new Date().toISOString()
  });
};
```

3. Password Utilities

```
// utils/password.js
import bcrypt from 'bcryptjs';

export const hashPassword = async (password) => {
  const saltRounds = 12;
  return await bcrypt.hash(password, saltRounds);
};

export const comparePassword = async (password, hashedPassword) => {
  return await bcrypt.compare(password, hashedPassword);
};
```

4. JWT Utilities

```
// utils/jwt.js
import jwt from 'jsonwebtoken';

export const generateAccessToken = (userId) => {
  return jwt.sign(
```

```
    { userId },
    process.env.JWT_SECRET,
    { expiresIn: process.env.JWT_EXPIRES_IN || '24h' }
  );
};

export const generateRefreshToken = (userId) => {
  return jwt.sign(
    { userId },
    process.env.JWT_REFRESH_SECRET,
    { expiresIn: '7d' }
  );
};

export const verifyToken = (token, secret = process.env.JWT_SECRET) => {
  return jwt.verify(token, secret);
};
```

5. Email Utilities

```
// utils/email.js
import nodemailer from 'nodemailer';

const transporter = nodemailer.createTransporter({
  host: process.env.SMTP_HOST,
  port: process.env.SMTP_PORT,
  secure: false,
  auth: {
    user: process.env.SMTP_USER,
    pass: process.env.SMTP_PASS
  }
});

export const sendOTPEmail = async (email, otp, firstName) => {
  const mailOptions = {
    from: process.env.FROM_EMAIL,
    to: email,
    subject: 'Email Verification - College Prep Platform',
    html: `
      <div style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto;">
        <h2>Welcome to College Prep Platform!</h2>
        <p>Hi ${firstName},</p>
        <p>Thank you for registering. Please verify your email address using the OTP below:</p>
        <div style="background-color: #f4f4f4; padding: 20px; text-align: center; margin: 20px 0;">
```

```
        <h1 style="color: #333; font-size: 32px; margin: 0;">${otp}</h1>
      </div>
      <p>This OTP will expire in 10 minutes.</p>
      <p>If you didn't create an account, please ignore this email.</p>
      <hr>
      <p style="font-size: 12px; color: #666;">College Prep Platform Team</p>
    </div>
  `
 };

 return await transporter.sendMail(mailOptions);
};

export const generateOTP = () => {
 return Math.floor(100000 + Math.random() * 900000).toString();
};
```

6. Code Execution Service Integration

```
// utils/codeExecution.js
export const executeCode = async (code, language, problemId, testCases) => {
 try {
   const response = await fetch(process.env.CODE_EXECUTION_SERVICE_URL + '/execute',
{
     method: 'POST',
     headers: {
       'Content-Type': 'application/json',
       'Authorization': `Bearer ${process.env.CODE_EXECUTION_API_KEY}`
     },
     body: JSON.stringify({
       code,
       language,
       problemId,
       testCases,
       timeLimit: 1000, // 1 second
       memoryLimit: 256 // 256 MB
     })
   });

   const result = await response.json();

   return {
     success: response.ok,
     data: result
```

```
    };
  } catch (error) {
    console.error('Code execution service error:', error);
    return {
      success: false,
      error: 'Code execution service unavailable'
    };
  }
};

export const getLanguageConfig = (language) => {
  const configs = {
    python: {
      extension: 'py',
      compileCommand: null,
      runCommand: 'python3 main.py',
      timeout: 5000
    },
    cpp: {
      extension: 'cpp',
      compileCommand: 'g++ -o main main.cpp -std=c++17',
      runCommand: './main',
      timeout: 3000
    },
    java: {
      extension: 'java',
      compileCommand: 'javac Main.java',
      runCommand: 'java Main',
      timeout: 5000
    },
    javascript: {
      extension: 'js',
      compileCommand: null,
      runCommand: 'node main.js',
      timeout: 5000
    }
  };

  return configs[language] || configs.python;
};
```

This completes the comprehensive Low-Level Design for the Home Page, Tracks, and Profile sections of your college placement preparation platform. The LLD includes:

1. **Detailed Database Schemas** - Complete MongoDB schemas with all fields and relationships
2. **Comprehensive API Endpoints** - Request/response structures with proper payload formats
3. **Frontend Components** - Detailed component structure with data requirements
4. **Authentication & Middleware** - Security, validation, and rate limiting
5. **Utility Functions** - Helper functions for common operations

Each section provides the exact technical specifications needed to implement the platform, including:

- Precise field names and data types
- API endpoint structures with headers and responses
- Frontend component hierarchy and data flow
- Error handling and validation logic
- Authentication and authorization mechanisms

This LLD serves as a blueprint for implementing a scalable, maintainable platform that addresses the core problem of bridging the gap between seniors and juniors for placement preparation.